

Modality Matters: A Transient Behavioral Interruption Rescues Agent WANDERING Where Residual Steering Does Not

Anonymous Authors
(submitted for double-blind review)

May 31, 2026

Abstract

A multi-turn coding agent fails in a distinctive way we call **WANDERING**: it keeps acting — reading, editing, running commands — but never emits the **finish** action, exhausting its turn budget. A companion line of work detects WANDERING from a residual-stream signature and a probe-free tool-entropy signal, but three pre-registered attempts to *intervene* on the residual stream (injecting a SUCCESS-direction at the output layer L55 and at the locus L11 that best discriminates WANDERING) all fail to rescue it. We ask whether the missing lever is not the locus but the *modality*. On the same 20 WANDERING Qwen3.6-27B SWE-bench Pro trajectories, gated by the same live tool-entropy collapse detector, we contrast four interventions that differ only in modality: no intervention; residual L11 injection; a content-neutral message; and a behavioral re-plan message. Analyzed paired (mandatory under run-instability), the behavioral interruption **roughly doubles the finalization rate**, 30% → 70% (McNemar $p = 0.021$), while residual injection is inert (40% vs. 30%, $p = 0.63$). Crucially the lever is the *interruption, not its content*: a neutral message that explicitly tells the agent to keep going rescues exactly as well as the re-plan message ($B0 \equiv B1$, $p = 1.0$). SWE-bench Pro Docker evaluation indicates the rescued finalizations are real fixes and that the interruption also raises the task solve-rate ($\sim 23\% \rightarrow 50\%$, paired 5–0, $p = 0.062$, cross-session) — a suggestive secondary effect. The result completes a four-paper arc (detect → localize → residual fails → behavioral works): for long-horizon agents, the predictive signal lives in the residual stream but the causal lever lives in behavior.

1 Background

WANDERING. Give a capable model a real software issue, a shell, and a fixed turn budget. Most runs end in one of two ways: the agent submits a patch via a **finish** tool (SUCCESS), or it locks onto a dead end and gives up early (LOCKED). A third failure is quieter: the agent keeps working turn after turn and never decides it is done, exhausting the budget without ever calling **finish**. We call this WANDERING. It is expensive (the full budget is consumed) and silent (the agent looks busy throughout), which makes it a natural target for monitoring and intervention.

The detector, and the mechanism hypothesis. The Tool-Entropy companion paper [1] shows WANDERING is detectable: in its final turns a WANDERING agent’s distribution over tool-call names *collapses* (low Shannon entropy), and this signature generalizes across architectures. That paper proposed a mechanism (its §13): the mid-layer “verdict” that the task is done is consolidated, but the edge circuits that translate the verdict into the **finish action** fail to fire — a decision-to-action desynchronization. The multi-channel classification paper [3] independently localizes the strongest WANDERING discriminator to an early, edge layer (L11) by stability selection.

The residual nulls. If the mechanism is “mid-layer ready, edge fails to emit,” the obvious fix is to inject the SUCCESS-typical edge-layer residual into a WANDERING trajectory to supply the missing translation. The residual-intervention paper [2] pre-registers and runs exactly this, three times: a forced-finish counterfactual (does WANDERING already secretly succeed? — no, Fisher $p = 0.71$); always-on SUCCESS-donor injection at the output layer L55 (inert, Fisher $p = 1.00$); and, re-targeted to L11 — the layer [3] flags as WANDERING’s strongest discriminator — a norm-matched magnitude sweep. The L11 injection is *not* inert (it destabilizes the model into malformed tool calls at high magnitude) but it does *not* rescue (paired McNemar $p = 0.73$). Correcting the locus from L55 to L11 did not convert the null. The conclusion of [2] is sharp: the residual signal that *detects* WANDERING is not a residual *lever* that fixes it, even at the strongest detector locus.

This paper. The residual nulls leave one variable unexplored: the intervention *modality*. All three prior attempts perturb the residual stream. Here we hold the locus question fixed (we intervene at the same detector-gated turn) and vary only *how* we intervene — a residual activation edit versus a behavioral message at the action interface. We ask: is the missing lever behavioral?

2 Related Work

The knowledge-action gap is now consensus — for single-forward-pass settings. Basu et al. [4] name the “knowledge-action gap”: linear probes detect model errors at near-perfect AUROC while four intervention paradigms (concept-bottleneck steering, SAE feature steering, activation patching, task-vector steering) correct almost none of them. Bhalla et al. [5] name the “predict/control discrepancy”: the features that predict a behavior are not the ones that control it. We do not claim the asymmetry as novel; we cite into it. What is unexamined in this literature is the asymmetry *on long-horizon agents*, where the failure is a multi-turn behavioral mode rather than a single-token error, and — critically — whether a *behavioral* lever closes the gap the residual lever cannot.

Behavioral course-correction of agents. SWE-PRM [6] course-corrects looping SWE agents at inference time with a process reward model, and its failure taxonomy (redundant exploration, looping, failure to terminate) overlaps WANDERING; it establishes that *behavioral* intervention can rescue looping agents. Our contribution is not behavioral-rescue per se — it is the controlled *modality contrast*: behavioral rescue *where residual injection fails*, on the same trajectories at the same trigger, plus the finding that the rescue is the interruption and not its content. SafeThink [7] shows a transient early intervention suffices for safety recovery (a structural analog, but activation-based and not agent-failure). EAST [8] shows activation steering *can* move an agent’s exploration entropy — so our residual null is specifically about *finalizing*, not about whether activations move agent behavior at all. StepShield [9] formalizes the “intervention gap” (time from first problematic behavior to intervention) that our detector-gated design optimizes. The tool-entropy signal we trigger on is independently validated as a model-agnostic quality signal [10]. The nearest method-competitor reads pre-action agent activations to predict tool need/risk but is detection-only and does not steer [11]. Our own prior work supplies the detector [1], the localization [3], and the residual nulls [2].

3 Method

Population. The 20 WANDERING trajectories from [2, 3]: Qwen3.6-27B on SWE-bench Pro, 50-turn budget, temperature 1.0, captured at L11/L23/L31/L43/L55. All runs use the deterministic per-instance seed. Hardware: one RTX 6000 Pro Blackwell (96 GB).

Live trigger. We fire one intervention per trajectory, online, at the first turn ≥ 10 at which the Shannon entropy of the last-10 tool-call names drops below $\tau = 0.503$ — the operating threshold of the deployed v5 detector [1]. Computed from the running trajectory’s own tool history (no oracle), the trigger fires on 18 of 20 instances in the live runs (19/20 in calibration on the original no-hook traces) at a median turn of 17, leaving roughly 30 turns for the intervention to act. This mimics deployment: the detector flags the collapse, and we intervene.

Four arms, varied only in modality. At the trigger turn we apply one of:

- **Arm 0** — nothing (no-intervention baseline).
- **Arm A (residual)** — attach a SUCCESS-donor injection at L11, $\alpha = 0.70$, `mode=add`, for the remaining turns (the lever of [2], here detector-gated).
- **Arm B0 (behavioral, neutral)** — inject one `user` message of matched length with content-neutral text: *“Here is a brief checkpoint note for your records. You may continue working on the task as you see fit; no change of approach is required. Proceed with whatever next step you judge appropriate.”*
- **Arm B1 (behavioral, re-plan)** — inject one `user` message: *“You appear to be repeating similar tool calls without converging. Stop and re-plan: in 2–3 sentences state what you have already established, then either (a) call `finish_tool` with your best patch now, or (b) name the single concrete blocker and the one next action to resolve it.”*

B0 is the behavioral analog of a placebo: it controls for “a message appeared / the context was refreshed / a turn was consumed,” isolating the re-plan *content*. All four arms run in the same session with the same seeds, so they share the pre-trigger trajectory and diverge only at the intervention.

Analysis: paired, never definitional. WANDERING is not run-stable under temperature sampling ([2]: the same 20 instances flip `finish` $\sim 35\%$ on a no-hook re-run). The only valid finalization test is therefore paired against the same-session Arm 0 (McNemar on discordant pairs), never against the definitional “0/20 all-WANDERING” baseline. We pre-register three hypotheses: **H1** $B1 > 0$ (does behavioral rescue?), **H2** $B1 > A$ (does modality matter?), **H3** $B1 > B0$ (is it the content?).

Solve-rate. Finalizing is the WANDERING-specific behavior, but a `finish` on a wrong patch is not a real fix. We therefore also score patch correctness with the official SWE-bench Pro Docker harness (apply the model patch to the base commit in a per-instance image, run the gold FAIL_TO_PASS and PASS_TO_PASS tests). Patches are computed as `git diff HEAD` where HEAD already contains the committed gold-test patch, so the scored diff excludes the gold tests (no leakage).

4 Results

4.1 Finalization: behavioral rescues, residual is inert

Arm	<code>finish</code> (rescue)	<code>max_turns</code>	<code>invalid_tools</code>
0 — no intervention	6/20 (30%)	14/20	0
A — residual L11 ($\alpha=0.70$)	8/20 (40%)	12/20	0
B0 — behavioral, neutral	14/20 (70%)	6/20	0
B1 — behavioral, re-plan	14/20 (70%)	6/20	0

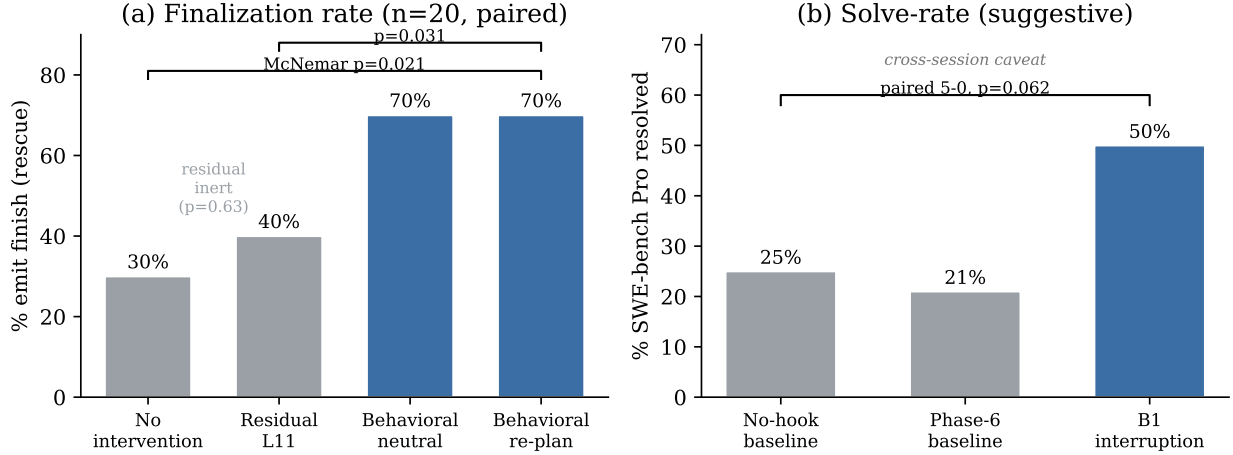


Figure 1: **Modality matters.** (a) Finalization rate by intervention arm on the same 20 WANDERING trajectories, gated by the same live tool-entropy collapse detector. The two behavioral arms (blue) roughly double the rate at which agents emit `finish` (30% $\rightarrow$ 70%); the residual L11 injection (gray) is inert (A vs. 0, $p = 0.63$). A content-neutral message rescues as well as a re-plan ($B0 \equiv B1$, $p = 1.0$): it is the interruption, not the content. (b) SWE-bench Pro solve-rate. The behavioral interruption (B1, 50%) strictly dominates two independent no-intervention baselines ($\sim 23\%$; paired 5-0, $p = 0.062$) — a suggestive secondary effect carrying a cross-session caveat (§4).

The behavioral arms roughly double the finalization rate (30% $\rightarrow$ 70%); the residual arm barely moves it (Figure 1a). Paired McNemar ($n=20$; the ≤ 2 instances that never trigger receive no intervention and are concordant by construction, so restricting to the triggered subset gives identical discordant counts):

- **H1 (B1 vs. 0):** 9 B1-only flips vs. 1 baseline-only, **McNemar** $p = 0.021$ — **PASS**. The behavioral interruption causally increases finalization.
- **H2 (B1 vs. A):** 6 B1-only vs. 0 A-only, $p = 0.031$ — **PASS**. Modality matters: behavioral beats residual.
- **A vs. 0:** 3 vs. 1, $p = 0.63$ — **null**. Residual injection does not beat baseline (Arm A’s 40% is run-instability, not a residual effect), replicating [2].
- No arm produces `invalid_tools`: the behavioral nudge does not destabilize the model (unlike the residual L11 hook at high magnitude in [2]).

We verified the finalization analysis two independent ways: a re-computation from the raw per-run JSON and the notebook’s own analysis cell agree exactly on every discordant count and p -value.

4.2 The lever is the interruption, not the content

H3 (B1 vs. B0): 2 B1-only vs. 2 B0-only flips, **McNemar** $p = 1.0$ — **the content-neutral message rescues exactly as well as the re-plan**. This is the sharpest single finding. B0 explicitly tells the agent that *no change of approach is required* and never mentions finishing, yet it doubles finalization just as B1 does. So the rescue is not the agent following good advice or being

told to finish — it is the *interruption itself*: a single fresh **user** turn at the collapse point breaks the tool-loop. WANDERING, behaviorally, is a loop that any context disruption can dislodge; the residual nudge cannot.

4.3 Solve-rate: a suggestive course-correction (with a transparency caveat)

Do the rescued finalizations resolve the issue, or does the interruption make agents submit garbage? The B1 patches resolve $10/20 = 50\%$ of issues (SWE-bench Pro Docker, authoritative `eval_results.json`). They are substantial diffs (1.7–17 KB, none empty), so the finalizations are real fixes, not hollow containment.

To ask whether the interruption *improves* solve-rate, we need a no-intervention baseline solve-rate. Here we must be transparent about a design defect: our runner wrote each patch to `{instance_id}.patch`, keyed by instance and not by arm, so the four arms overwrote one another and only the last arm’s (B1’s) patches survived. We therefore cannot compute a *same-session* per-arm solve-rate. We instead use two *independent* no-intervention baselines that do have saved patches: the paper-2 no-hook re-run of the same 20 WANDERING ($5/20 = 25\%$) and the original Phase-6 WANDERING patches scored in [2] ($4/19 = 21\%$). The two agree at $\sim 23\%$, indicating solve-rate is far more run-stable than finalization. Against this:

Condition	solve-rate	note
No-hook baseline (same 20)	$5/20 = 25\%$	paper-2 session
No-hook baseline (Phase 6)	$4/19 = 21\%$	[2]
B1 (behavioral interruption)	$10/20 = 50\%$	this work

Paired on the same 20 instances, B1 *strictly dominates* the no-hook baseline: it resolves all 5 instances the baseline resolves *plus 5 more*, with zero regressions (5 B1-only vs. 0 baseline-only). McNemar $p = 0.062$; unpaired Fisher $p = 0.19$. We label this **suggestive**: the effect is a clean doubling with a strictly-dominant direction, but it is marginal at $n=20$ and — because of the patch-overwrite defect — the B1 and baseline patches come from *different sessions*. (Finalization, by contrast, is a same-session comparison because Arm 0’s *finalization* was recorded in the run log even though its *patch* was lost.) A same-session per-arm solve-rate would confirm it; the fix is a one-line per-arm patch save, deferred to a confirmatory replication.

A finer point. Within B1, solve-rate is the same whether the agent finalized or not (`finish` $7/14 = 50\%$, `max_turns` $3/6 = 50\%$). So the interruption’s quality improvement is partly independent of its finalization improvement: it helps the agent write a better patch *and* declare done, and the two effects do not require each other. This is why we read the result as a behavioral *course-correction*, richer than the pure “unblock the emit” reading of the §13 mechanism.

5 Discussion

Modality matters. Holding the locus and the trigger fixed and varying only how we intervene, a behavioral message at the collapse turn rescues WANDERING (finalization $p = 0.021$; solve-rate suggestively doubled) while a residual activation edit at the same turn does not (replicating three prior residual nulls). The asymmetry the field has named for single-forward-pass settings [4, 5] thus has a constructive resolution *for long-horizon agents*: the predictive signal lives in the residual stream, but the causal lever lives in behavior — at the action interface, not the representation. This completes the four-paper arc: detect (residual/tool-entropy) → localize (L11) → residual intervention fails → behavioral intervention works.

Why a static residual direction cannot steer a 50-turn process. A WANDERING trajectory is a high-variance stochastic process (the 35% run-instability of [2]). A single always-on additive direction nudges one sample of that process; a fresh `user` turn re-anchors the whole rollout. The interruption is closer to the natural control surface of an agent — its inputs — than any residual edit.

It is the interruption, not the content. That a neutral, “carry on” message rescues as well as a structured re-plan (H3 null) is mechanistically informative: WANDERING is dislodgeable by disruption rather than instruction. For deployment this is good news — the cheapest possible intervention (one templated turn, no model call, no advice) suffices to break the loop — and it cautions against over-crediting the *content* of course-correction methods.

Deployment value vs. offline score. SWE-bench scores the final diff regardless of whether `finish` was called, so finalization per se does not move an offline leaderboard. But a WANDERING agent that never finalizes is useless in production: it burns the full budget, never returns a result, and never signals completion. The interruption makes it terminate earlier (median finish turn 34 of the 50-turn budget), return, and — suggestively — solve more. The value is in the deployment loop, which is precisely the agent-monitoring setting our detector targets.

6 Limitations

Sample and scope. $n = 20$ WANDERING, single model (Qwen3.6-27B), single benchmark (SWE-bench Pro). The finalization effect is significant ($p = 0.021$) but modest- N ; cross-architecture replication (e.g. Llama-70b) is the natural robustness check.

Solve-rate is cross-session and marginal. Because of the patch-overwrite defect (§4), the solve-rate baseline comes from different runs than B1; the comparison is suggestive ($p = 0.062$), not confirmed. We report it transparently rather than omit it, and we do not claim a significant solve-rate gain. The one-line fix (save patches per arm) enables a same-session confirmation.

One intervention class. We test a single transient `user` message and a single always-on residual direction. We do not test transient residual pulses, sustained behavioral scaffolding, or tool-space restriction (a fourth behavioral arm we scoped but did not run).

Trigger threshold. The trigger uses the deployed v5 threshold $\tau = 0.503$; we did not sweep it. Coverage (19–20/20) was robust to $\tau \in [0.503, 0.70]$ in calibration, so the result is not threshold-fragile, but a sweep would map the precision/recall of the gate.

7 Conclusion

On the same WANDERING trajectories where residual steering fails three times, a transient behavioral interruption — one fresh `user` turn at the detector-gated collapse point — roughly doubles the rate at which agents finalize (30% $\rightarrow$ 70%, paired McNemar $p = 0.021$), and the lever is the interruption itself rather than its content (a neutral message rescues as well as a re-plan). SWE-bench Pro evaluation indicates the rescued finalizations are real fixes and suggests the interruption also doubles the task solve-rate ($\sim 23\% \rightarrow 50\%$, cross-session, $p = 0.062$). For long-horizon agents the predictive signal is residual but the causal lever is behavioral: detection does not imply a steerable direction, but it does point to a steerable *action*.

Future Work

(1) Same-session per-arm solve-rate (the one-line patch-save fix) to upgrade the solve-rate from suggestive to confirmed. (2) Cross-architecture replication (Llama-70b). (3) A v5-gated transient pulse vs. the always-on residual direction — whether a behavioral-*timed* residual edit narrows the modality gap. (4) Tool-space restriction as a fourth behavioral arm. (5) Whether the “interruption, not content” finding generalizes to other agent loop-failures beyond WANDERING.

Reproducibility

The experiment is fully pre-registered (design, frozen prompts, and decision gate committed before the behavioral arms were collected). The detector gate, the four-arm intervention loop, the 80 per-arm run records, the finalization analysis (independently re-computed from the raw per-run logs), and the SWE-bench Pro patch-pass evaluation are released; finalization figures are reproduced from the raw logs by a single script. Anonymized for review.

References

- [1] Anonymous. Tool-Entropy Collapse: A Cross-Architecture Signature of Agent WANDERING Failure. Zenodo, 2026. DOI 10.5281/zenodo.20368601.
- [2] Anonymous. Causal Localization of Agent WANDERING to Edge-Layer L11: The Right Locus Is Still Not a Rescue Lever. Companion paper, 2026.
- [3] Anonymous. Multi-Channel Mechanistic Signatures of Agent WANDERING: Classification, Causal Localization, and Behavior-Legible Rescuability. Companion paper, 2026.
- [4] Basu et al. Interpretability without actionability: mechanistic methods cannot correct LM errors despite near-perfect internal representations. arXiv:2603.18353, 2026.
- [5] Bhalla et al. Towards Unifying Interpretability and Control: Evaluation via Intervention. ICLR 2025 (arXiv:2411.04430).
- [6] Gandhi et al. When Agents Go Astray: Course-Correcting SWE Agents with PRMs. arXiv:2509.02360, 2025.
- [7] Ghosal et al. SafeThink: Safety Recovery in Reasoning Models Is Only a Few Early Steering Steps Away. arXiv:2602.11096, 2026.
- [8] Rahn, D’Oro, Bellemare. Controlling LLM Agents with Entropic Activation Steering. arXiv:2406.00244, 2024.
- [9] Felicia et al. StepShield: When, Not Whether to Intervene on Rogue Agents. arXiv:2601.22136, 2026.
- [10] Li et al. Rethinking the Role of Entropy in Optimizing Tool-Use Behaviors for LLM Agents. arXiv:2602.02050, 2026.
- [11] Tatsat & Shater. Beyond the Black Box: Interpretability of Agentic AI Tool Use. arXiv:2605.06890, 2026.

A Per-instance outcomes

Table 1 lists, for each of the 20 WANDERING instances, the finalization outcome of each arm (F=finish, M=max_turns; no invalid_tools occurred) and whether the no-hook baseline and the B1 patches resolved the SWE-bench Pro issue. A reviewer can recompute every paired McNemar count directly from this table.

Table 1: Per-instance finalization (arms 0/A/B0/B1) and solve (baseline vs. B1).

Instance	0	A	B0	B1	base	B1
ansi-3db08a	M	M	F	M	✓	✓
ansi-4c5ce5	M	M	M	M	–	–
ansi-502270	F	F	F	F	✓	✓
ansi-5d253a	F	M	F	M	–	✓
ansi-a26c32	M	M	M	F	–	✓
ansi-cd9c4e	F	F	F	F	✓	✓
ansi-de01db	F	F	F	F	–	✓
open-09865f	F	F	F	F	–	✓
open-5c6c22	M	F	F	F	–	–
open-5de7de	M	M	M	M	–	–
open-7f6b72	M	M	F	F	–	–
open-bb152d	M	M	F	F	–	–
open-c05ccf	M	F	F	F	–	✓
qute-0d2afd	M	M	F	F	–	–
qute-0fc6d1	M	M	M	F	–	–
qute-2dd896	F	F	F	F	–	–
qute-85b867	M	M	M	M	✓	✓
qute-990291	M	M	M	M	–	–
qute-9b71c1	M	M	F	F	–	–
qute-cf06f4	M	F	F	F	✓	✓
finish/solve	6	8	14	14	5	10